코드 설명 및 결과 분석 보고서

휴머노이드 예비단원 이주원

1. 서론

2. 코드 분석
   - 설정
   - Q네트워크
   - 리플레이 버퍼와 학습 설정
   - 행동 선택 함수
   - 학습
   - 학습 루프

3. 결과 분석
   - 실험 조건
   - 탐험 비율 엡실론
   - 학습 보상
   - 평가 보상
   - TD loss
   - 무작위 정책과 비교
   - 타겟 복사 주기 변경 실험
   - 전체적인 분석

4. DQN설명
   - Q러닝에서 DQN으로
   - 리플레이 버퍼와 target 네트워크의 역할
   - TD target 수치 예제

1. 서론

이 과제는 DQN을 이용하여 CartPole 환경을 학습시키는 것이다. 카트폴의 상태는 카트 위치, 카트 속도, 막대 각도, 막대 각속도 4개의 소수 값이라 표로 저장할 수 없다. 그래서 이번 과제에서는 Q테이블 대신 신경망으로 Q값을 구하는 DQN을 사용하는것이다.

이 코드는 크게 총 8개의 부분으로 나누어져 있다. 첫번째로 라이브러리와 시드를 설정하고 카트폴 환경을 만드는 부분이다. 두번째로 Q네트워크를 만드는 부분이다. 상태 4개를 입력받아 행동 2개의 Q값을 출력하는 신경망을 학습용과 정답 계산용으로 두 개 만든다. 세번째로 리플레이 버퍼와 학습 설정 부분이다. 경험을 저장할 공간을 만들고 optimizer와 loss 함수를 정한다. 네번째로 행동 선택 함수이다. 엡실론 그리디 방식으로 탐험과 활용 중 하나를 골라 행동을 결정한다. 다섯번째로 한 번의 학습을 하는 함수이다. 저장된 경험 중 64개를 무작위로 뽑아 정답과 예측값의 차이로 online 네트워크를 갱신한다. 여섯번째는 학습 루프이다. 4만 스텝 동안 환경과 상호작용하며 경험을 쌓고, 4스텝마다 학습하고, 500스텝마다 target 네트워크를 갱신한다. 일곱번째는 모델 저장과 평가 부분이다. 학습한 모델을 파일로 저장한 뒤 다시 불러와 탐험 없이 50회 평가한다. 여덟번째는 학습된 모델이 실제로 막대를 세우는 모습을 화면으로 보여주는 부분이다.

2. 코드분석

2-1. 설정

짐나심으로 카트폴 환경을 만들고, torch로 신경망을, numpy로 계산을 한다. random, numpy, torch, 환경의 시드를 모두 0으로 고정하여 실행할 때마다 같은 결과가 나오게 하였다.

2-2. q네트워크

make_net은 입력 4개를 받아 은닉층 64개, 64개를 거쳐 행동 2의 Q값을 출력하는 신경망을 만든다. 은닉층 사이에는 ReLU를 사용한다. 같은 구조의 네트워크를 online과 target 두 개 만든다. 온라인은 실제로 학습하는 네트워크이고, 타겟은 정답을 계산할 때만 쓰는 복사본이다. 타겟은 처음에 온라인과 같은 값으로 시작하고, requires_grad_(False)로 직접 학습되지 않게 하였다.

2-3. 리플레이 버퍼와 학습 설정

memory는 경험을 최근 20,000개까지 저장하는 리플레이 버퍼이다. 가득 차면 가장 오래된 경험부터 지워진다. optimizer는 Adam, 학습률은 0.0005이고, loss 함수는 SmoothL1Loss를 사용한다.

2-4. 행동 선택 함수

select_action은 엡실론 그리디 방식으로 행동을 고른다. 무작위 수가 엡실론보다 작으면 무작위로 행동하고, 그렇지 않으면 온라인 네트워크에 상태를 넣어 q값이 더 큰 행동을 고른다. 행동을 고를 때는 학습이 아니므로 torch.no_grad()로 기울기를 계산하지 않는다.

2-5. 학습

optimize_batch는 메모리에서 경험 64개를 무작위로 뽑아 한 번 학습한다. 41~45줄에서 상태, 행동, 보상, 다음 상태, 종료 여부를 각각 텐서로 바꾼다. 47줄에서 온라인 네트워크로 실제 했던 행동의 q값을 구한다. 49~50줄에서 타겟 네트워크로 정답 y를 계산한다.

52줄에서 예측값과 정답의 차이로 손실을 구하고, 53~56줄에서 Day1과 같이 zero_grad, backward, step 순서로 온라인 네트워크를 갱신한다. 55줄은 한 번에 가중치가 너무 크게 바뀌지 않도록 기울기 크기를 10으로 제한하는 부분이다.

2-6. 학습 루프

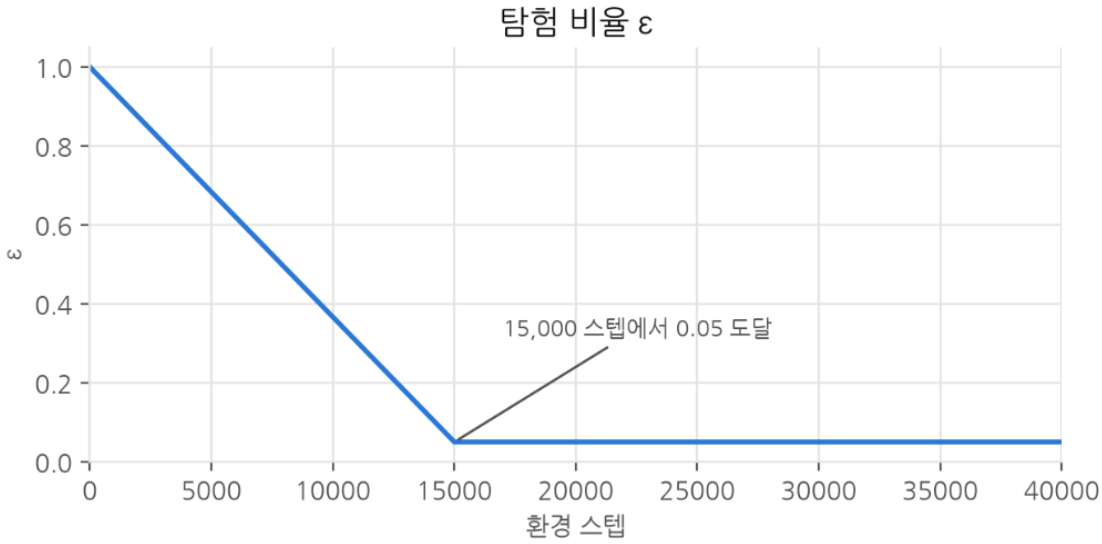
총 40,000 스텝 동안 학습한다. 60줄에서 엡실론은 1에서 시작해 15,000 스텝까지 0.05로 일정하게 줄어들고, 그 뒤로는 0.05로 유지된다. 매 스텝 행동을 하고 그 결과를 메모리에 저장한다. 메모리에 경험이 1,000개 이상 모이면 4스텝마다 한 번 학습하고(64~65줄), 500스텝마다 온라인 네트워크의 값을 타겟 네트워크로 복사한다(66~67줄). 에피소드가 끝나면 환경을 다시 시작한다.

3. 결과 분석

3-1. 실험 조건

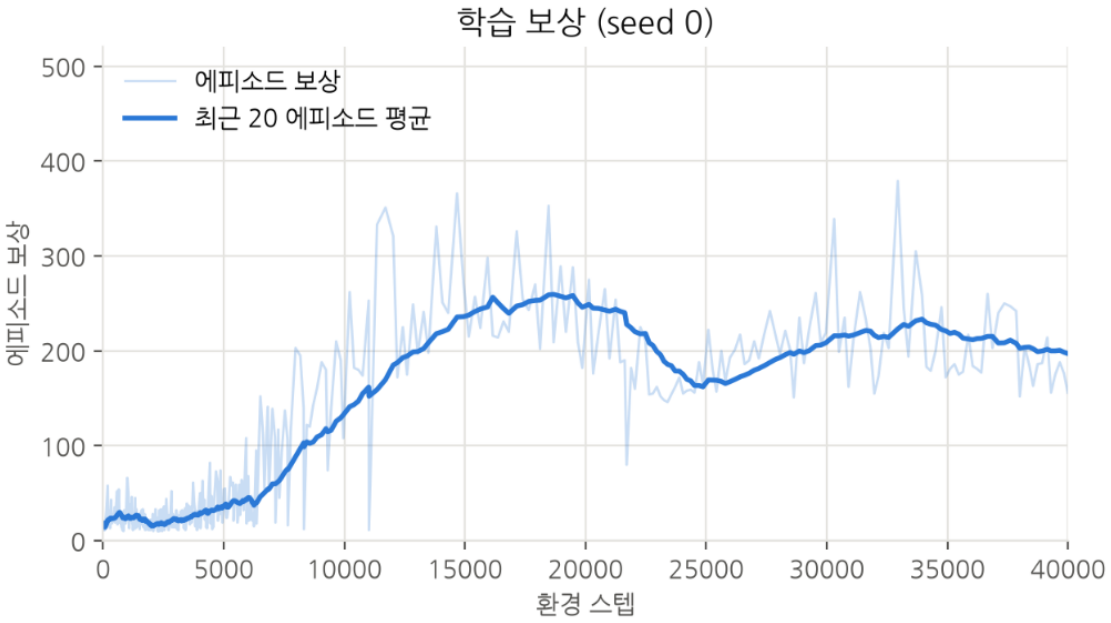
변인 통제를 위해 시드를 0으로 고정하고 40,000 스텝 학습하였다. 학습 중에는 2,000 스텝마다 탐험 없이 10회씩 평가하였고, 학습이 끝난 뒤에는 저장한 모델을 다시 불러와 시드 1000부터 50회 평가하였다. 타겟 복사 주기 실험은 결과가 운에 따라 달라지는지 확인하기 위해 시드 0, 1, 2로 세 번씩 실행하였다.

3-2. 탐험 비율 엡실론



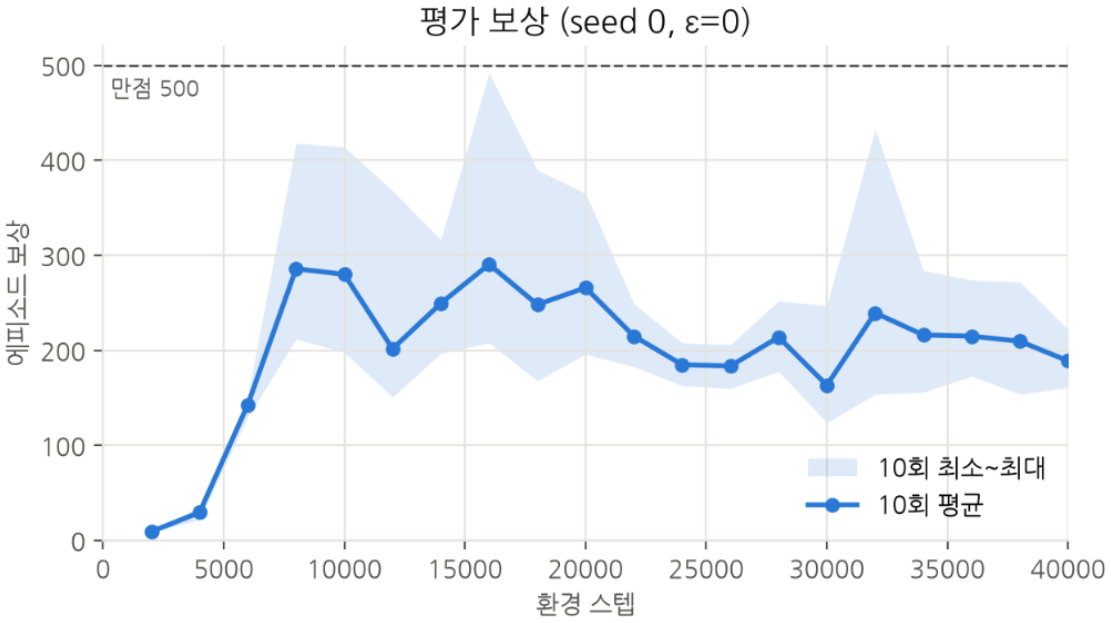
엡실론은 만 오천 스텝까지 1에서 0.05로 일정하게 줄어들고, 그 뒤로는 0.05로 유지된다. 그래서 15000 스텝 전의 학습 보상에는 무작위 행동이 많이 섞여 있다. 예를 들어 6천 스텝에서는 엡실론이 약 0.65로, 행동의 절반 이상을 무작위로 하고 있다. 15,000 스텝 이후에도 20번 중 1번은 무작위로 행동하므로, 학습 중 보상은 실제 실력보다 조금 낮게 나온다.

3-3. 학습 보상



6천 스텝까지는 보상이 20~40 정도로 무작위로 움직일 때와 비슷하다. 아직 엡실론이 크고 메모리에 쌓인 경험도 적기 때문일걸로 보인다. 그 뒤로 빠르게 올라 16,000~19,000 스텝에서 최근 20 에피소드 평균이 약 255로 가장 높았다. 그런데 21000~25000 스텝에서 약 165까지 떨어졌다가 다시 220 정도로 회복하였고, 마지막에는 약 200이었다. 이렇게 잘하다가 갑자기 못하게 되는 이유는 q테이블과 신경망의 차이 때문으로 보인다. q테이블은 한 칸의 값을 고치면 그 칸만 바뀐다. 하지만 신경망은 가중치를 고치면 비슷한 상태들의 q값이 같이 바뀐다. 그래서 새로운 것을 배우다가 전에 잘하던 것을 잊어버릴 수 있다. 학습 중 가장 높은 보상은 약 380이었고, 끝까지 만점 500에는 도달하지 못했다.

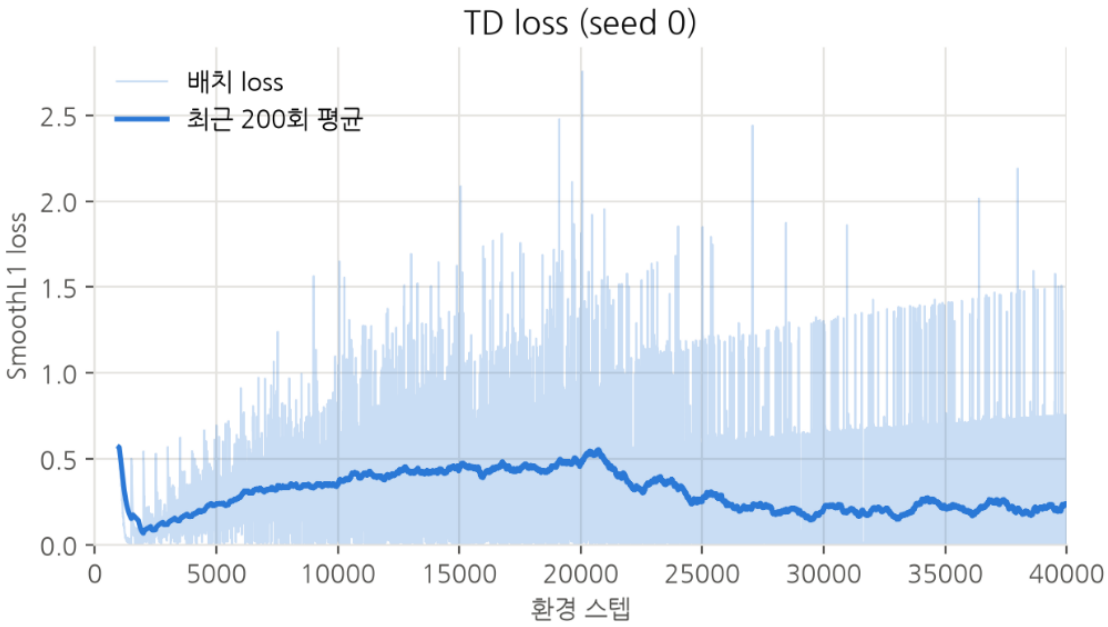
3-4. 평가 보상



평가는 엡실론을 0으로 두어 탐험 없이 q값이 큰 행동만 고르게 하였다. 가장 좋았던 시점은 16,000 스텝으로 평균 290.6이었다. 그 이후로는 160~240 사이를 오르내리다가 마지막에는 약 190이었다. 즉 끝까지 학습한 모델이 가장 좋은 모델이 아니었다. 평가 점수가 가

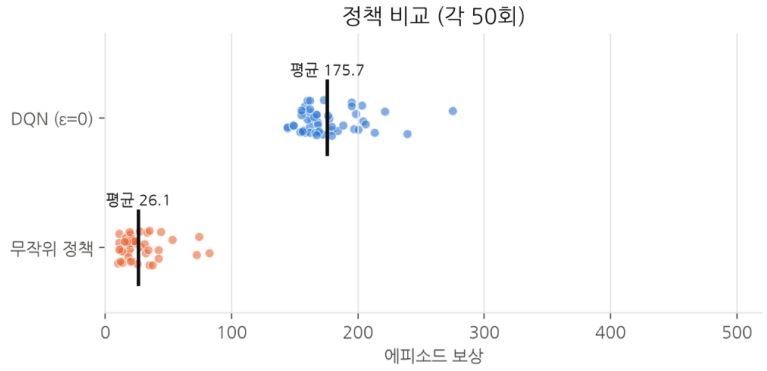
장 높을 때의 모델을 따로 저장해두었다면 더 좋은 결과를 얻었을 것이다.
또 같은 시점의 10회 평가끼리도 최소와 최대가 100 넘게 차이 나는 경우가 많았다. 시작 위치가 조금만 달라도 결과가 크게 달라질 만큼 아직 정책이 확실하게 자리 잡지 못했다는 뜻이다.

3-5. TD 손실



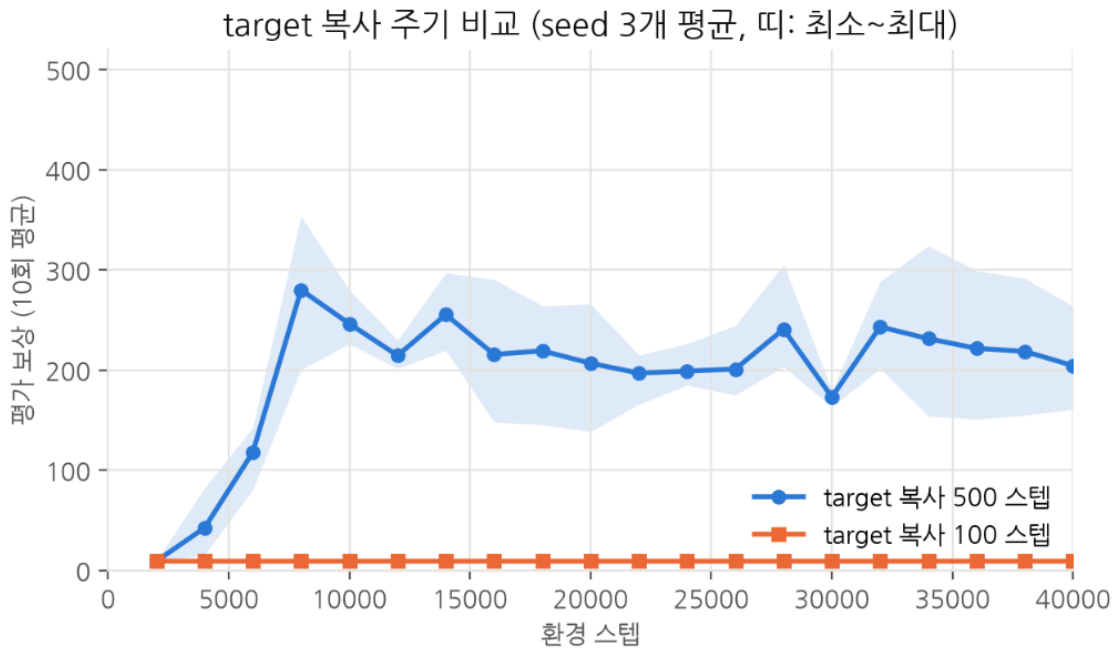
손실은 메모리에 경험이 1,000개 모인 뒤부터 기록되었다. 처음에 크게 떨어진 뒤, 이만 스텝까지 0.1에서 0.5로 오히려 커졌다. MNIST에서는 손실이 줄어들면 학습이 잘 된 것이었지만 DQN은 다르다. 엠니스트는 정답이 정해져 있지만, 디큐엔의 정답 y는 타겟 네트워크가 계산하기 때문에 학습이 진행될수록 정답 자체가 바뀐다. 또 에이전트가 오래 버틸수록 q값이 커지고(학습이 끝났을 때 약 50~56), 그만큼 오차도 커진다. 그래서 DQN에서는 손실만 보고 학습이 잘 되었는지 판단하기 어렵고, 보상 그래프를 같이 봐야 한다.
배치 손실 그래프에서 튀는 값들은 다음 절의 막대가 넘어진 경험 때문으로 보인다. 이 경험은 오차가 약 48이고, SmoothL1 손실로 약 48이 된다. 한 번에 64개를 뽑아 평균을 내므로, 이런 경험이 하나만 섞여도 배치 손실이 약 48 ÷ 64 = 0.75가 된다. 그래프 후반에 0.75 근처에 띠가 생긴 것과 일치한다. 1.5 근처 값은 이런 경험이 두 개 섞인 경우로 보인다.

3-6. 무작위 정책과 비교



그래프에서 두 점 무리가 전혀 겹치지 않는다. DQN이 가장 못했을 때도 무작위 정책이 가장 잘했을 때보다 높다. 따라서 학습된 정책은 어떤 경우에도 무작위보다 잘한다고 할 수 있다. 다만 50회 중 500을 채운 적은 한 번도 없었다.

3-7. 타겟 복사 주기 변경 실험



500 스텝에서는 세 시드 모두 학습에 성공했지만, 시드에 따라 175.7에서 253.8까지 차이가 컸다. 그래프의 파란 띠가 넓은 것도 이 때문일것이다.
100 스텝에서는 세 시드 모두 처음부터 끝까지 약 9.4에 머물렀다. 무작위 정책보다도 낮다. 무작위로 움직이면 왼쪽과 오른쪽을 섞어서 밀기 때문에 어느 정도 버티지만, 한쪽으로만 계속 밀면 9~10 스텝 만에 막대가 넘어진다. 즉 네트워크가 망가져서 항상 같은 쪽으로만 밀게 된 것이다. Day1 엡니스트에서 학습률을 너무 크게 했을 때 모든 이미지를 1로 예측했던 것과 비슷하다.
원인은 정답이 너무 자주 바뀌었기 때문으로 보인다. 학습은 4스텝마다 한 번이므로, 500 스텝 주기는 학습 125번마다, 100 스텝 주기는 학습 25번마다 정답을 바꾸는 것이다. 정답을 쫓아가는 도중에 정답이 계속 바뀌니까 제대로 따라가지 못하고, 결국 q값이 한쪽으로 치우쳐 학습이 무너진 것으로 보인다.

3-8. 전체적인 분석

DQN은 무작위보다 확실히 잘하는 정책을 학습했지만, 학습 중간에 성능이 떨어졌다가 다시 오르는 등 불안정했다. 마지막 모델이 가장 좋은 모델이 아니었으므로, 평가 점수가 가장 높을 때 모델을 저장하는 것이 좋다. 손실은 MNIST와 달리 학습이 잘 되었는지 판단하는 기준으로 쓰기 어려웠다. 또 타겟 복사 주기 하나만 바꿔도 학습이 완전히 실패할 수 있어서, 타겟 네트워크가 dqn에서 꼭 필요하다는 것을 확인하였다. 성능을 더 올리려면 학습 스텝을 늘리거나, 복사 주기를 1,000 스텝처럼 더 길게 해보는 실험이 필요해 보인다.

4. DQN 설명

4-1. q러닝에서 dqn

프로즌레이크는 상태가 16개뿐이라 16×4 크기의 q테이블에 q값을 저장할 수 있었다. 하지만

카트폴의 상태는 막대 각도 0.0123처럼 소수로 된 값이라 칸을 나눌 수 없다. 그래서 q테이블 대신 신경망을 쓴다. 상태를 넣으면 행동별 q값이 나오는 신경망이 q테이블 역할을 하는 것이다.
갱신하는 방법도 달라진다. q러닝은 q테이블의 값을 직접 정답 쪽으로 조금씩 옮겼다. dqn은 정답 y와 예측한 q값의 차이를 손실로 두고, backward와 step으로 신경망의 가중치를 고친다.

4-2. 리플레이 버퍼와 타겟 네트워크의 역할

리플레이 버퍼는 지금까지 겪은 경험을 모아두는 저장소이다. 바로 이어진 스텝들은 막대 각도나 카트 위치가 거의 비슷하다. 이것을 겪은 순서대로 바로 배우면, 최근에 겪은 상황에만 맞춰져서 전에 배운 것을 잊어버리기 쉽다. 그래서 경험을 쌓아두고 무작위로 섞어서 64개씩 뽑아 배운다. 또 한 번 겪은 경험을 여러 번 다시 쓸 수 있어서 적은 경험으로도 많이 배울 수 있다.
타겟 네트워크는 정답 y를 계산하는 데만 쓰는 온라인 네트워크의 복사본이다. 만약 학습 중인 온라인 네트워크로 정답을 계산하면, 가중치를 고칠 때마다 정답도 같이 바뀐다. 타겟 네트워크는 500 스텝 동안 그대로 있다가 한 번씩 온라인 네트워크의 값으로 복사되기 때문에, 그동안은 정답이 고정되어 안정적으로 배울 수 있다. 복사 주기를 100 스텝으로 줄였을 때 학습이 완전히 실패한 것이 타겟 네트워크가 왜 중요한지 보여준다.

4-3. TD target 수치 예제

정답은 다음 식으로 계산한다.
y = r + 0.99 × (1 − terminated) × 다음 상태의 최대 q값
SmoothL1은 오차가 클 때 제곱하지 않고 그대로 쓴다. 제곱 오차로 계산했다면 손실이 커져 이 경험 하나가 학습을 크게 흔들었을 것이다. SmoothL1을 쓰면 이런 큰 오차가 있어도 학습이 덜 흔들린다.
q값 56이 무슨 뜻인지도 계산해볼 수 있다. 매 스텝 보상 1을 받고 0.99씩 할인하면, 약 82 스텝을 더 버틸 때 합이 56이 된다. 즉 네트워크는 이 상태에서 약 82 스텝 정도 더 버틸 수 있다고 예상하고 있다.